Highlights

Two-scale stochastic simulation of epidemics on community networks with aggregate inter-community coupling

Yulian Kuryliak, Michael T. M. Emmerich

- Exact local Gillespie engines are coupled through aggregate inter-community hazards
- The ideal scheme is exact simulation of a surrogate all-to-all interface network
- Aggregate export clocks are stochastically early for randomly placed interfaces
- Accuracy is organised by internal timing spread relative to edge crossing time
- Close arrival times can coexist with physically impossible transmission routes

Two-scale stochastic simulation of epidemics on community networks with aggregate inter-community coupling

Yulian Kuryliak[a,*], Michael T. M. Emmerich[b]

[a]*Institute of Computer Science and Information Technologies, Lviv Polytechnic National University, 12 Stepan Bandera Street, Lviv, 79000, Ukraine*

[b]*Faculty of Information Technology, University of Jyväskylä, P.O. Box 35 (Agora), Jyväskylä, FI-40014, Finland*

ARTICLE INFO

Keywords:
Stochastic simulation
Multiscale simulation
Gillespie algorithm
Aggregate coupling
Model fidelity
Community networks

Abstract

Multiscale stochastic simulation often couples detailed subsystem dynamics through aggregate interfaces. We study this problem for epidemic spreading on community networks, where within-community contact structure is represented explicitly while inter-community coupling is available only in aggregate form. Each community is evolved by a Gillespie engine that is exact conditional on the imported events it receives; the macro layer generates imports from aggregate, state-dependent hazards and, in the default implementation, places them uniformly among susceptible nodes of the receiving community. We give a complete specification of the method, including its grid-based synchronisation, and show that its ideal continuous-time version is an exact simulation of a surrogate network in which each bundle of inter-community edges is replaced by uniform all-to-all coupling of equal total weight. This representation identifies four sources of discrepancy: loss of boundary-state information, uniform import placement, absence of interface-edge exhaustion, and synchronisation error. For uniformly random interfaces in the single-export setting, we further show that the ideal aggregate export clock is stochastically earlier than the full-network one. Controlled experiments on synthetic SI networks compare the method with full-network Gillespie simulation in single-export benchmarks and in 48 configurations of ten-community chains (50–1000 nodes per community, internal path lengths 1–6, one or five interface edges, 400 matched runs each). When the spread of infection times within a community was below a quarter of the mean single-edge crossing time, terminal arrival-time distributions were statistically indistinguishable in 15 of 18 configurations; beyond this regime, the two-scale simulator arrived early by an amount roughly proportional to that ratio, reaching 40%. Close timing did not imply admissible routes: up to 52% of aggregate crossings preceded infection of any true bridge node.

*Corresponding author

yulian.a.kuryliak@lpnu.ua (Y. Kuryliak); michael.t.m.emmerich@jyu.fi (M.T.M. Emmerich)

ORCID(s): 0000-0002-6600-2637 (Y. Kuryliak); 0000-0002-7342-2090 (M.T.M. Emmerich)

1. Introduction

Epidemics spread at more than one scale. Inside a community, infection may pass through a detailed network of contacts between people (Keeling and Eames, 2005; Pastor-Satorras et al., 2015; Kiss et al., 2017). Between communities, the available data are often much less detailed. For example, they may give only passenger numbers, commuting flows, or a weighted link between two places (Balcan et al., 2009; Barbosa et al., 2018). We therefore need a model that can use a detailed network inside each community and simpler data between communities.

This situation is common in practice. A local contact network may be observed, reconstructed, or generated to match known properties such as clustering and mean shortest-path length (Eubank et al., 2004; Danon et al., 2011). In contrast, links between communities may be known only through transport, migration, commuting, or mobile-phone data (Barbosa et al., 2018; Chang et al., 2021). Such data can tell us the total strength of connection between two communities, but not which people form the connection. We may therefore know the total coupling without knowing the bridge nodes or their infection states.

If every contact is known, a continuous-time Markov chain (CTMC) can describe the epidemic, and a Gillespie-type method can simulate it exactly (Gillespie, 1977; Gibson and Bruck, 2000). Efficient methods are also available for temporal, large, and sparse networks (Vestergaard and Génois, 2015; Kuryliak et al., 2025). However, these methods still need the individual contacts that may transmit infection. Metapopulation models use aggregate flows between communities, but usually also replace the detailed local network by population counts (Colizza et al., 2007; Colizza and Vespignani, 2008; Balcan et al., 2010; Ajelli et al., 2010). Our problem lies between these two cases: we want to keep node-level stochastic spread inside every community while using only aggregate information between communities.

The basic two-scale simulator was presented in an earlier workshop paper (Kuryliak and Emmerich, 2025). It runs one stochastic local simulator for each community. A separate macro layer uses the current community states and the aggregate coupling to generate infections that enter from another community. The present article gives a full description of this method and studies its approximation error. In particular, we ask what is lost when real cross-community edges and their endpoints are replaced by an aggregate connection, and what is lost when an imported infection is placed on a selected susceptible node.

Our main question is: when does this aggregate interface reproduce the behaviour of the full-network CTMC well enough for the quantity of interest? We study arrival times, their distributions, and the physical validity of the transmission route. In particular, similar arrival times do not prove that the route was physically possible: the aggregate model can export infection before any real bridge node has been infected. The complete microscopic network is known in our controlled experiments, so it can be used as a reference. The two-scale simulator receives only the local networks and aggregate connections. These experiments test the method; they are not a calibration of one particular epidemic.

This article makes four contributions.

1. We give a complete description of the two-scale simulator, including the time-grid scheduler used in the experiments (Section 3, Algorithm 1).
2. We show exactly what the ideal continuous-time method simulates. It is exact for a *surrogate network*: the real cross-community edges are replaced by weak all-to-all connections with the same total weight (Proposition 1). It is not, in general, exact for the original physical network. We then identify the resulting sources of error and analyse a single export (Proposition 2).
3. We compare the method with full-network Gillespie simulation for single exports and chains of communities. Permutation baselines show how much difference can arise from finite sampling alone (Sections 5 and 6).
4. We give an empirical indicator of accuracy for our benchmarks. It compares the spread of infection times inside a community with the typical time needed to cross one interface edge (Section 6.6).

We consider susceptible–infected (SI) dynamics on static synthetic networks and use one fixed synchronisation step. Section 2 explains the relation to earlier methods. Section 3 describes the simulator. Section 4 explains what the aggregate interface preserves and loses. Sections 5 and 6 present the experiments and results. Section 7 discusses use, cost, and limitations.

2. Related work

Simulation of stochastic events. Gillespie's direct method draws the next event and its time from the current event rates (Gillespie, 1977). The next reaction method keeps a separate clock for each possible event (Gibson and Bruck, 2000). Both methods are exact for the CTMC they simulate. When an event rate changes with time, its rate can be accumulated until it reaches a random threshold (Ethier and Kurtz, 1986). Anderson (2007) used this idea for systems with time-dependent rates and delays. Our macro clock uses the same idea, but its rate changes because the local epidemic states change. Network simulators can save work by updating only the rates affected by the latest event (Vestergaard and Génois, 2015; Kuryliak et al., 2025). Our local simulators use the implementation of Kuryliak et al. (2025).

Some multiscale methods speed up chemical-reaction models by approximating reactions that occur much faster than others (Haseltine and Rawlings, 2002; Cao et al., 2005). Our method removes a different kind of detail. It keeps the local infection events, but does not know which individual cross-community edges are currently able to transmit. Even so, the relative speeds of local spread and cross-community transmission help explain its error.

Models with several communities. Metapopulation models represent each subpopulation by counts and connect the subpopulations by mobility flows. They have been used to study global spread and the conditions for invasion (Colizza et al., 2007; Colizza and Vespignani, 2008; Balcan et al., 2009, 2010). Detailed mobility data can also be combined with local transmission models (Chang et al., 2021). Agent-based models instead represent individuals throughout the model (Eubank et al., 2004; Kerr et al., 2021). Ajelli et al. (2010) compared these two broad approaches. The method studied here keeps individual nodes and contacts inside each community, but replaces the unknown individual contacts between communities by one aggregate stochastic interface.

Two levels of mixing and co-simulation. Some epidemic models combine contacts in small local groups with uniform mixing at a wider level (Ball et al., 1997). Our surrogate network is related to this idea, but uniform mixing is used only between connected pairs of communities, and its strength is set by their aggregate weight.

The implementation also has features of co-simulation. In co-simulation, separate subsystem solvers move forward on a shared time grid and exchange a limited amount of information (Gomes et al., 2018). Here, the local community simulators are those subsystems. They send their infected and susceptible counts to the macro layer. In this implementation, those exchanged counts are used to draw a cross-community infection time and then to choose its receiving community and target node. The scheduler can therefore treat the local simulators as black boxes.

3. The two-scale simulation method

The experiments use one implementation of the two-scale simulator, with a time grid for coordinating the communities. To understand its error, we also consider the process that the same rates would define if every event time were calculated exactly. We call this theoretical description the *ideal process*. It separates the error caused by changing the network from the additional error caused by the time grid. The ideal process is exact for the surrogate network defined later; this does not make it exact for the original physical network. Below, we explain the reference model, the available data, and the steps of the implemented simulator.

3.1. Full-network reference model

Let $G = (V, E)$ be the contact network. Its nodes are divided into M non-overlapping communities:

$$V = V_1 \cup V_2 \cup \cdots \cup V_M, \qquad N_i = |V_i|. \tag{1}$$

Here, the sets V_i do not overlap, and N_i is the number of nodes in community i. We write E_{ii} for contacts inside community i, and

$$E_{ij} = \{(u, v) \in E : u \in V_i,\, v \in V_j\}, \qquad i \neq j,$$

for contacts from community i to community j. We treat each contact as an oriented transmission channel. An undirected contact therefore gives two channels, one in each direction.

Every node is susceptible (S), infected (I), or removed (R). Its full state at time t is $X(t)$, and $X_i(t)$ is the part of this state in community V_i. In community i, let $S_i(t)$ and $I_i(t)$ be the sets of susceptible and infected nodes. Their sizes are $s_i(t)$ and $y_i(t)$, and their fractions are $\bar{s}_i(t) = s_i(t)/N_i$ and $\bar{y}_i(t) = y_i(t)/N_i$. For a state x, the notation $x^{v:I}$ means the same state after node v becomes infected, and $x^{u:R}$ means the state after node u is removed.

The full model follows two simple rules. A susceptible node v receives infection from each infected neighbour u at rate β_{uv}. An infected node is removed at rate γ. We can write both rules as

$$q_v(x) = \sum_{\substack{u:x_u=I \\ (u,v)\in E}} \beta_{uv} \quad \text{for } x_v = S, \qquad r_u(x) = \gamma \quad \text{for } x_u = I. \tag{2}$$

Thus, $q_v(x)$ is the total infection rate of a susceptible node v, and $r_u(x)$ is the removal rate of an infected node u; the corresponding rate is zero for nodes in other states. A rate describes how often an event occurs: for a constant rate q, the mean waiting time is $1/q$. These rates also define the CTMC generator $\mathcal{L}_G$ used later; no additional epidemic rule is hidden in that notation. For a cross-community channel, we write $\beta_{uv} = \beta_{\text{out}} w_{uv}$, where w_{uv} is its contact weight. We call exact simulation of these full-network rates Full. All experiments in this article use SI dynamics, so $\gamma = 0$.

3.2. Aggregate interface

The two-scale simulator, denoted TwoScale, knows every local graph $G_i = (V_i, E_{ii})$ and its rates. It does not know the individual cross-community channels in E_{ij}. For each ordered pair of communities, it knows only the total weight

$$W_{ij} = \sum_{(u,v)\in E_{ij}} w_{uv}, \qquad i \neq j. \tag{3}$$

If all weights equal one, W_{ij} is the number of directed channels from V_i to V_j. In an application, this total may instead come from transport, migration, mobility, or contact data. The macro layer also receives the current susceptible and infected counts from each local simulator. It does not know which nodes are endpoints of real cross-community edges. Table 1 gives the main difference, and Figure 1 shows it visually.

Information or action	FULL	TWOSCALE
Local contacts, rates, and node states	known	known by local simulators
Individual cross-community edges and bridge nodes	known	unknown
Total cross-community weight W_{ij}	obtained from the edges	given directly
Cross-community event rate	uses states of real edge endpoints	uses $\bar{y}_i$, $\bar{s}_j$, and W_{ij}
Target of a cross-community infection	endpoint of the active edge	a susceptible node drawn by the target rule
Number of possible SI infections through one bundle	at most the number of its channels	at most s_j target infections

Table 1: Information used by the two models. The "target rule" in TWOSCALE is uniform among susceptible nodes in all experiments in this article.

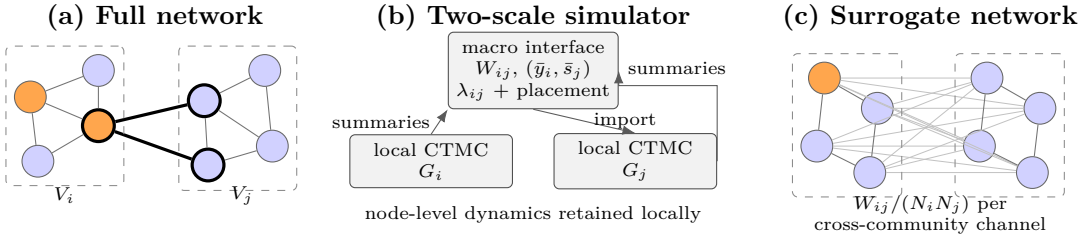


Figure 1: Three views of the interface. (a) In the full network, infection can cross only through the shown physical edges and their bridge nodes. (b) The two-scale simulator keeps a separate local simulator for each community. The macro layer receives community counts, uses W_{ij} to generate a cross-community infection, and places it on a uniformly selected susceptible node. (c) The ideal two-scale process is equivalent to a surrogate network in which every ordered node pair across the two communities has a weak connection. The connections together have total weight W_{ij}. Orange nodes are infected and blue nodes are susceptible.

3.3. Local simulators

Each community has a separate Gillespie simulator for its local graph G_i. It applies the infection and removal rates in (2) using only the local edges E_{ii}. The implementation updates only rates affected by the latest event, following Kuryliak et al. (2025). Each local simulator has its own random-number stream.

The macro layer can ask a local simulator to move to a specified time. The simulator applies the local events up to that time. The macro layer can then import an infection by changing one selected susceptible node to I. This changes the local rates. The implementation keeps the unused part of the random threshold for the next local event and recalculates its waiting time using the new total rate. It then selects the event type and node using the updated rates. A new local threshold is drawn only if there is no existing one. Each local clock has its own threshold, separate from the macro threshold described in Section 3.5. For prescribed import times and target nodes, the local simulator follows the CTMC rules of its own network. The approximation comes from the way the macro layer generates and places imports.

3.4. Macro layer

The macro layer represents each community by its current fractions and represents each directed connection by W_{ij}. Its rate for an infection from community i to community j is

$$\lambda_{ij}(t) = \beta_{\text{out}} W_{ij} \bar{y}_i(t) \bar{s}_j(t), \qquad \Theta(t) = \sum_{i \neq j} \lambda_{ij}(t). \tag{4}$$

The factors have a direct meaning: $\beta_{\text{out}} W_{ij}$ gives the total connection strength, $\bar{y}_i$ is the infected fraction in the source, and $\bar{s}_j$ is the susceptible fraction in the receiving community. The sum Θ is the rate of any cross-community infection.

When such an event occurs, first select the source and receiving communities (i, j) with probability λ_{ij}/Θ. Then choose one susceptible node uniformly in the receiving community j and infect it. If there are 20 susceptible nodes, each has probability 1/20. This selection does not use real bridge nodes, because the aggregate interface does not know them. In later formulas, we write this target probability as $\Pi_{ij}(v \mid x) = 1/s_j$ for $v \in S_j$, where x is the current network state. Thus, Π simply denotes the probability of choosing a particular susceptible node. Other target rules can be used if bridge-node information is available. The optional calibration factor in the implementation is one in all experiments.

This aggregate rate is generally an approximation to the rate across the real physical edges. For fixed community counts, it is the expected rate if the endpoints of those edges are placed uniformly at random, before their identities, states, and epidemic history are known. It becomes the actual rate for the uniform all-to-all surrogate interface described in Section 4. The rate Θ changes after any local event or imported infection that changes a community count.

3.5. Timing and synchronisation

After a macro event at time t_0, draw a random threshold $E \sim \mathrm{Exp}(1)$: an exponential random number with mean one. Starting from zero, accumulate the total macro rate Θ. The next macro event occurs at the first time t^* when

$$\int_{t_0}^{t^*} \Theta(s)\, ds = E. \tag{5}$$

In words, the area under the rate curve grows until it first reaches the random threshold. If it never reaches the threshold, no further macro event occurs. This is the standard cumulative-rate construction (Ethier and Kurtz, 1986; Anderson, 2007). If Θ is constant, the waiting time is E/Θ. Drawing $E = -\ln U$, with U uniform on $(0, 1)$, gives the familiar Gillespie waiting time. When Θ changes after a local event, we keep the same threshold and continue accumulating with the new rate. We draw a new macro threshold only after a macro event. This is the cumulative-rate clock used in the workshop description. Exact calculation of its first crossing defines the ideal process; the time-grid calculation below approximates that crossing. Section 4.1 shows that the ideal process is exact for the surrogate network.

The implemented scheduler checks progress on intervals of length τ. The workshop description also uses a midpoint estimate for this accumulated-rate clock. Here, we specify how the estimate is obtained from the local simulators. Let A be the amount already accumulated since the previous macro event. For the current interval $[t, t_g)$, copy every local simulator, including its random-number state, and advance each copy to the midpoint. The copies give an estimate Θ_{mid} of the total macro rate. The estimated increase over the remaining interval is $\Theta_{\mathrm{mid}}(t_g - t)$. If A plus this increase is below E (the random macro threshold), the real simulators move to t_g. Otherwise, estimate the event time by

$$t^* = t + \frac{E - A}{\Theta_{\mathrm{mid}}}. \tag{6}$$

Here, E is the random macro threshold and A is the amount already accumulated toward it. The numerator $E - A$ is what remains; dividing by the estimated rate gives the remaining waiting time. For example, if $E = 1$, $A = 0.4$, and $\Theta_{\mathrm{mid}} = 0.2$, the estimated wait is three time units. The scheduler advances the real simulators to t^*, selects the source and receiving communities using their states at that time, chooses the susceptible target, and imports the infection. Algorithm 1 gives the full sequence.

Algorithm 1 Grid-synchronised two-scale simulation of one realisation.

Input: local graphs G_i, weights W_{ij}, rates, step τ, horizon T_{end}, and initial state.

1. Set $t = 0$ and accumulated rate $A = 0$; draw $E \sim \text{Exp}(1)$.
2. While $t < T_{\text{end}}$ and infection can still spread, set $t_g = \min(t + \tau, T_{\text{end}})$ and process the interval up to t_g.
3. Copy every local simulator, including its random-number state. Advance the copies to the midpoint of the remaining interval and calculate Θ_{mid} from (4).
4. If $A + \Theta_{\text{mid}}(t_g - t) < E$, advance the real simulators to t_g, add this amount to A, set $t = t_g$, and return to step 2.
5. Otherwise, calculate t^* from (6) and advance the real simulators to t^*.
6. Select (i, j) with probability $\lambda_{ij}(t^*)/\Theta(t^*)$. Select v uniformly from $S_j(t^*)$ and import infection at v. Update the receiving local simulator, keeping its unused exponential threshold and selecting the local event from the new rates.
7. Set $t = t^*$ and $A = 0$, draw a new $E \sim \text{Exp}(1)$, and process the rest of the current interval in the same way.

The copied random-number states are important. Unless an import occurs first, a copied simulator follows the same path that the real simulator will follow to the midpoint. The midpoint estimate is exact on an interval with no local event. Its timing error and its look-ahead of at most $\tau/2$ decrease to zero as τ decreases.

The time grid is needed because the scheduler treats local simulators as black boxes. If they also report their next local-event times, the scheduler can merge those times and accumulate the piecewise-constant value of Θ exactly. That event-driven version removes the numerical synchronisation approximation. The experiments in this article use the implemented grid version with $\tau = 1$.

3.6. Computational structure and intended use

Full stores all local and cross-community channels, and its work per event grows with node degree (Kuryliak et al., 2025). TwoScale stores the local graphs and an $M \times M$ matrix of aggregate weights. The workshop paper discusses this reduction in the information stored for cross-community contacts (Kuryliak and Emmerich, 2025). Its importance depends on the problem. Replacing many individual cross-community edges by one weight can reduce storage. If only a few such edges exist, this saving may be small, and the local graphs still have to be stored.

The separate local simulators also provide an opportunity for parallel execution. Once the scheduler has chosen the next target time (t_g or t^*), the local simulators can advance independently to that same time. The midpoint copies can also be processed in parallel. The macro layer must then collect the counts and apply any scheduled import before continuing. The implementation studied here advances the local simulators sequentially; parallel execution within one run is a possible extension, not a measured speedup. Local simulators can also use different community-specific settings.

The grid scheduler adds work by copying local states and coordinating the communities. Therefore, neither lower memory use nor faster execution is guaranteed. The method is useful when individual cross-community contacts are unavailable or too numerous to represent explicitly. Section 7.2 reports the measured run times; this article primarily evaluates approximation error.

4. What the aggregate interface preserves and what it loses

4.1. Surrogate-network representation

With exact timing (5), the ideal two-scale process is a CTMC on $\{S, I, R\}^V$. Its rate of the transition $x \to x^{v:I}$ by an import is $\sum_{i\neq j} \lambda_{ij}(x)\Pi_{ij}(v \mid x)$ for $v \in S_j(x)$; all local transitions have the rates of (2) restricted to E_{ii}. The following proposition identifies this process with a full-network process on a modified graph.

Proposition 1 (Surrogate network). *Let $\tilde{G}$ be the network with the same within-community channels and rates as G, in which every inter-community bundle E_{ij} is replaced by the complete set of channels $V_i \times V_j$, each with rate $\tilde{\beta}_{uv} = \beta_{\mathrm{out}} W_{ij}/(N_i N_j)$. Under uniform placement, the ideal two-scale process has generator $\mathcal{L}_{\tilde{G}}$; that is, it has the law of the full-network process on $\tilde{G}$ started from the same configuration.*

Proof. Removals and within-community infections have identical rates in both processes. Consider a susceptible node $v \in V_j$ in configuration x. Under uniform placement, the two-scale rate at which v is infected by an import is

$$\sum_{i\neq j} \lambda_{ij}(x)\,\frac{1}{s_j(x)} = \sum_{i\neq j} \beta_{\mathrm{out}} W_{ij} \frac{y_i(x)}{N_i}\frac{s_j(x)}{N_j}\frac{1}{s_j(x)} = \sum_{i\neq j}\sum_{u\in V_i} \frac{\beta_{\mathrm{out}} W_{ij}}{N_i N_j}\mathbf{1}\{x_u = I\},$$

which is the total rate of the inter-community channels of $\tilde{G}$ that point into v. When $s_j(x) = 0$ both rates vanish. Hence all transition rates coincide and the generators are equal. □

Corollary 1. *If every inter-community bundle of G is already complete, $E_{ij} = V_i \times V_j$, with a common rate per pair of communities, then $\tilde{G} = G$ and the ideal two-scale process is exact.*

The surrogate preserves the total weight of every bundle, $\sum_{(u,v)\in V_i\times V_j} \tilde{\beta}_{uv} = \beta_{\mathrm{out}} W_{ij}$, but spreads it uniformly over all pairs (Figure 1c). The approximation error of the ideal method is therefore the difference between epidemics on G and on $\tilde{G}$, which differ only in their interfaces. The implemented method adds a numerical synchronisation error. Proposition 1 also provides a practical verification route: for small networks, the synchronisation error can be isolated by comparing the two-scale simulator with exact simulation on $\tilde{G}$.

4.2. Sources of discrepancy

Comparing G with $\tilde{G}$ gives four mechanisms by which the two-scale simulator can depart from the full-network process.

Boundary-state information (source side). In G, the rate of transmission from V_i to V_j is

$$a^{\star}_{ij}(x) = \beta_{\mathrm{out}} \sum_{(u,v)\in E_{ij}} w_{uv}\,\mathbf{1}\{x_u = I,\, x_v = S\}, \tag{7}$$

which depends on whether the particular bridge nodes are infected. In $\tilde{G}$, every infected node of V_i contributes. Exports can therefore occur before any bridge node is infected. Such *bridge bypass* is impossible in G and provides a direct diagnostic (Section 4.3).

Placement (receiver side). In G, a transmission infects the target endpoint of the firing channel; in $\tilde{G}$, the target is uniform among susceptible nodes. The location of an import determines its distance to the next interface and hence later crossings.

Interface exhaustion (receiver side). Under SI dynamics, a channel of G transmits at most once: afterwards its target is infected. A bundle with $|E_{ij}|$ channels therefore carries at most $|E_{ij}|$ transmissions, and a single-edge interface exactly one. In $\tilde{G}$, imports from V_i into V_j continue at rate $\lambda_{ij} = \beta_{\mathrm{out}} W_{ij} \bar{y}_i \bar{s}_j$ until V_j is exhausted; their expected number over a period $[t_1, t_2]$ is $\mathbb{E}\int_{t_1}^{t_2} \lambda_{ij}(s)\,ds$. When a receiving community remains largely susceptible for a long time, the two-scale simulator therefore keeps seeding it with additional infections that the full network cannot deliver.

Synchronisation. The midpoint quadrature and interpolation of Section 3.5 approximate (5).

Remark 1 (Placement under randomised interfaces). Suppose that the endpoints of each bundle are sampled uniformly and independently of the internal graphs and of the endpoints of other bundles, as in the experiments below. Then the target of the first transmission into a fully susceptible community is uniformly distributed in G, because it is the endpoint of a uniformly placed channel, and also in $\tilde{G}$. Placement can therefore affect arrival times only through transmissions after the first. In such designs, differences between the first crossing and later crossings isolate receiver-side effects (Section 6.6).

4.3. Single export: stochastic ordering and bridge bypass

Consider a source community V_0 of size n with an arbitrary internal graph, SI dynamics, and a single susceptible leaf node ℓ joined to a set $B \subset V_0$ of k bridge nodes by channels of rate β_{out}. Then $W_{0\ell} = k$, and before the export $\bar{s}_\ell = 1$, so that $\lambda_{0\ell} = \beta_{\text{out}} k\, \bar{y}_0$. Let τ_u be the infection time of node u, $A_u(t) = (t - \tau_u)^+$ its infection age, and $T_B = \min_{u\in B} \tau_u$.

Proposition 2 (Aggregate export clocks are early). *Let B be uniformly distributed over the k-subsets of V_0, independently of the internal graph, the initial configuration and the event randomness. Let T^{Full} and T^{TwoScale} be the first infection times of ℓ in the full network and in the ideal two-scale process. Then*

$$\mathbb{P}(T^{\text{Full}} > t) \;\geq\; \mathbb{P}(T^{\text{TwoScale}} > t) \qquad \text{for all } t \geq 0,$$

so that T^{TwoScale} is stochastically smaller than T^{Full} (Shaked and Shanthikumar, 2007). In particular, the mean and every quantile of the ideal two-scale export time are no larger than those of the full-network export time.

Proof. Before ℓ is infected, it does not influence V_0: it transmits nothing, and the rates of source nodes depend only on their infected neighbours. Both export times can therefore be constructed on one probability space from an autonomous SI trajectory X_0 of the source community, continued beyond the export, and an independent threshold $E \sim \text{Exp}(1)$:

$$T^{\text{Full}} = \inf\Big\{t : \beta_{\text{out}} \sum_{u\in B} A_u(t) \geq E\Big\}, \qquad T^{\text{TwoScale}} = \inf\Big\{t : \beta_{\text{out}} \frac{k}{n} \sum_{u\in V_0} A_u(t) \geq E\Big\},$$

where the second integrand follows from $\int_0^t \bar{y}_0(s)\, ds = n^{-1} \sum_u A_u(t)$. Conditional on X_0, the survival probabilities are $\exp(-\beta_{\text{out}} \sum_{u\in B} A_u(t))$ and $\exp(-\beta_{\text{out}} (k/n) \sum_u A_u(t))$. Since B is uniform and independent of X_0, $\mathbb{E}[\sum_{u\in B} A_u(t) \mid X_0] = (k/n) \sum_u A_u(t)$, and Jensen's inequality for the convex function $z \mapsto e^{-\beta_{\text{out}} z}$ gives $\mathbb{P}(T^{\text{Full}} > t \mid X_0) \geq \mathbb{P}(T^{\text{TwoScale}} > t \mid X_0)$. Taking expectations completes the proof. □

The inequality is strict whenever, with positive probability, the infection ages differ between nodes at time t and $0 < k < n$. It also holds when B is uniform among the nodes other than the initially infected node, because excluding the oldest infected node can only lower the conditional mean of the bridge hazard. The size of the gap is controlled by the conditional variance of the bridge hazard, for which sampling without replacement gives

$$\text{Var}\Big(\beta_{\text{out}} \sum_{u\in B} A_u(t) \;\Big|\; X_0\Big) = \beta_{\text{out}}^2\, k\, \frac{n-k}{n-1}\, \text{Var}_u\big(A_u(t)\big), \tag{8}$$

where Var_u is the population variance over source nodes. Once the source is fully infected, $\text{Var}_u(A_u(t)) = \text{Var}_u(\tau_u)$. The aggregate clock is therefore close to exact when node infection times are concentrated relative to $1/\beta_{\text{out}}$, as in a rapidly saturating clique, and is early when infection spreads slowly and unevenly through the source.

The same construction yields the probability of bridge bypass. In G, no export can precede T_B. In the ideal two-scale process,

$$\mathbb{P}\big(T^{\text{TwoScale}} < T_B\big) = \mathbb{E}\Big[1 - \exp\Big(-\beta_{\text{out}} \frac{k}{n} \int_0^{T_B} y_0(s)\, ds\Big)\Big], \tag{9}$$

which is the probability that the aggregate hazard accumulated before the true interface becomes active triggers an export. It increases when the bridges are reached late and after many other nodes, which is the situation in sparse, long-path communities. Appendix A gives $\mathbb{E}[T_B]$ for a clique source.

4.4. A time-scale descriptor

Both source-side and receiver-side mechanisms act only while infection is still spreading inside a community. By (8), the source-side gap is governed by β_{out} times the dispersion of infection times. The expected number of surplus imports into a community during its internal spread is of order $\beta_{\text{out}} W_{ij}$ times the duration of that spread. Let σ_T denote the standard deviation of node infection times within a community, which measures this duration. We use the dimensionless ratio

$$\rho = \beta_{\text{out}}\, \sigma_T, \tag{10}$$

the internal timing spread in units of the mean single-edge crossing time $1/\beta_{\text{out}}$, as a descriptor of the expected accuracy. The two mechanisms scale differently with the interface width ($\sqrt{k}$ in (8) and k for surplus imports, partly offset because the full network itself admits up to k crossings). We therefore do not include k in ρ, and we test the descriptor empirically rather than derive an error law.

5. Experimental design

5.1. Benchmarks

Table 2 summarises the four experimental series. All use SI dynamics, unit contact weights, equal internal and external per-edge rates ($\beta_{\text{in}} = \beta_{\text{out}} = \beta$), and the scheduler of Algorithm 1 with $\tau = 1$. In the *clique–leaf* benchmark, a leaf is attached to $k = 2$ bridge nodes of a 100-node clique. The *source–leaf sweep* replaces the clique by connected random graphs of $n = 1000$ nodes with a prescribed average shortest-path length (ASPL). In the *chain* series, $L = 10$ communities are joined in a line by k edges between consecutive communities; infection starts in community 0. Connected random graphs are generated by adding uniformly random edges to a uniformly random labelled tree, with the number of added edges calibrated so that the ASPL lies within 0.1 of its target; ASPL 1 is the complete graph. Mean degree therefore decreases with the target ASPL. At a target ASPL of 6 it ranges from 2.08 for $n = 50$ to 3.4 for $n = 1000$. Interface endpoints are sampled uniformly, independently for each interface and replicate.[1] The initial infected node is pseudorandom and shared by both simulators, and it may be a bridge node; in the clique–leaf benchmark, the seed is a fixed node of the clique.

[1] In the source–leaf sweep, bridge sets are drawn from all nodes except one fixed label. Because the random graphs and the uniformly chosen seed are invariant in law under relabelling, this is equivalent to uniform placement, so the assumption of Proposition 2 holds.

Series	Structure	k	β	Runs per cell and model
Clique–leaf	source clique $n = 100$ + one leaf	2	0.005	10 000
Source–leaf sweep	source $n = 1000$, ASPL 1–6 in steps of 0.25, + one leaf	1, 5	0.003	400 (40 × 10)
Chain of cliques	$L = 10$ cliques of $n = 1000$	1	0.003	400 (40 × 10)
Chain size sweep	$L = 10$, $n \in \{50, 100, 250, 1000\}$, ASPL 1–6 in steps of 1	1, 5	0.003	400 (40 × 10)

Table 2: Experimental series. Each cell of a sweep uses 40 network realisations (blocks) with 10 interface and event replicates each. ASPL: target internal average shortest-path length (1 denotes the complete graph).

Full and TwoScale runs are matched: they share the graph, the interface and the initial node, but use independent event streams. All exports and arrivals entering the metrics were observed, so no censoring adjustment is needed. The horizon was 1000 for the clique–leaf benchmark and 30 000 otherwise. Two Full runs were extended to 40 000 to observe their terminal arrivals (Appendix C). The chain of cliques is the ASPL-1, $n = 1000$, $k = 1$ cell of the size sweep, reported separately because it is the most favourable case.

5.2. Observables and metrics

For leaf benchmarks, we record the export time T_{export} (first infection of the leaf) and the bridge-activation time T_B. For chains, T_c is the first infection time of community c, measured from the initial infection; the terminal arrival is T_9. We report three summaries:

- the relative median shift $b_c = 100\,[\text{med}(T_c^{\text{TwoScale}}) - \text{med}(T_c^{\text{Full}})]/\text{med}(T_c^{\text{Full}})$, where negative values mean earlier arrival in TwoScale;
- the relative Wasserstein-1 distance $e_W = 100\,W_1/\overline{T}^{\text{Full}}$, where $W_1 = R^{-1}\sum_r |T_{(r)}^{\text{TwoScale}} - T_{(r)}^{\text{Full}}|$ is computed from the order statistics of the R times of each model;
- the bridge-bypass fraction, i.e. the percentage of TwoScale crossings that occur before any bridge node of the source community is infected.

For chains, the *local* shift of crossing c compares the mean of $T_c - T_{c-1}$ between models. We contrast the first crossing with the pooled crossings 2–9, using $T_9 - T_1$.

5.3. Statistical procedures

Confidence intervals are 95% percentile intervals from a bootstrap that resamples whole network blocks, keeping their ten replicates together, with 2000 resamples for chain summaries. Because e_W is positive even when two samples come from the same law, we calibrate it by a within-pair permutation test: the labels of the two runs in each matched pair are swapped with probability 1/2, and e_W is recomputed 2000 times. Under the null hypothesis that matched runs are exchangeable, this yields a sampling floor, which we summarise by its 95th percentile, and a p-value for the observed e_W. Appendix B gives an analytical approximation of the floor that applies when per-run data are not available.

The event seeds of a replicate depend on the target ASPL, the network block and the replicate index, but not on n or k; seeds are therefore shared across community sizes and interface widths (common random numbers). This reduces the variance of comparisons between such cells, but it also correlates their sampling errors. We therefore do not count agreement between cells as independent replication.

5.4. Verification of the reference

For a clique source with uniformly placed bridges, the expected bridge-activation time is known in closed form (Appendix A). Across the four clique-chain sizes and both interface widths, the mean T_B in Full agreed with the formula to within 2.3 standard errors. For $k = 1$, the mean waiting time after bridge activation (326.5–354.0) agreed with its exact value $1/\beta = 333.3$ to within sampling error (Table 3).

6. Results

6.1. Single export from a clique

Export times from a rapidly saturating source agree closely, but the aggregate interface admits impossible transmission orders. In the clique–leaf benchmark (Figure 2), mean export times were 110.18 (FULL) and 112.62 (TWOSCALE); the matched difference of +2.43 had a bootstrap interval of $[-0.35, 5.27]$. The empirical CDFs nearly coincide ($W_1 = 3.25$, i.e. $e_W = 2.95\%$; Kolmogorov–Smirnov distance 0.0142, asymptotic $p \approx 0.27$). Of the mean export time of about 110, only $\mathbb{E}[T_B] = 8.36$ is needed on average to infect the first bridge node (11); the remainder is the wait for transmission across the interface, at a rate of at most $k\beta = 0.01$. The period in which boundary and community-wide states differ is therefore short, and the aggregate clock is close to the exact one, as Proposition 2 and (8) suggest.

The ordering diagnostic nevertheless separates the models: in 98 of 10 000 TWOSCALE runs (0.98%), the leaf was infected before either bridge node, which never happens in FULL (Figure 2b,c). The middle panel shows $T_{\text{export}} - T_B$, a physical waiting time in FULL but a signed ordering diagnostic in TWOSCALE.

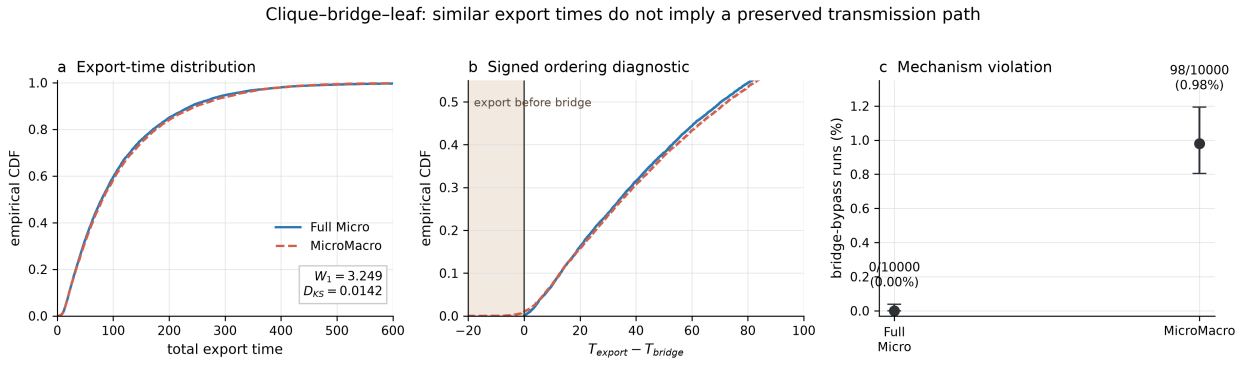


Figure 2: Clique–leaf benchmark ($n = 100$, $k = 2$, $\beta = 0.005$, 10 000 runs per model). (a) Empirical CDFs of the export time. (b) Empirical CDFs of $T_{\text{export}} - T_B$; negative values (shaded) are exports before any bridge node is infected. (c) Percentage of such bridge-bypass runs with 95% intervals. "Full Micro" denotes FULL and "MicroMacro" denotes TWOSCALE.

6.2. Single export from connected random sources

As internal paths lengthen, bridge bypass becomes common and the aggregate export becomes early, as predicted by Proposition 2. Figure 3 shows the source–leaf sweep. Mean bridge activation times were similar in the two models, within 7% at every ASPL (e.g. 1165 and 1136 at ASPL 6, $k = 1$), as expected from identical local dynamics before the export. In FULL with $k = 1$, the wait after bridge activation stayed at 322–359 at every ASPL, consistent with Exp(β) with mean 333. In TWOSCALE, the signed interval $T_{\text{export}} - T_B$ ($k = 1$) shrank from about 340 in dense sources to 207–295 at ASPL ≥ 5, because aggregate hazard accumulated before the true bridge became infectious.

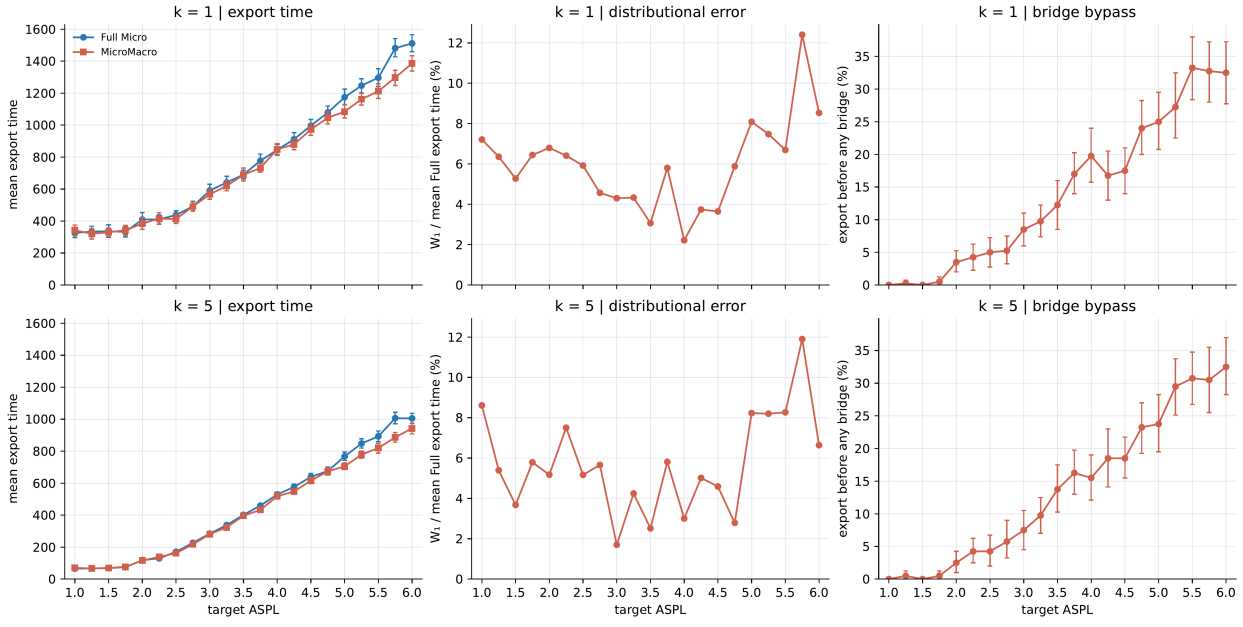


Figure 3: Source–leaf sweep: $n = 1000$, $\beta = 0.003$, $k = 1$ (top) and $k = 5$ (bottom), 400 matched runs per point. Left: mean export time. Centre: relative Wasserstein distance e_W of the export-time distributions. Right: percentage of TwoScale exports that precede the first bridge infection; Full had none. Whiskers are 95% block-bootstrap intervals. "Full Micro" denotes Full and "Production MicroMacro" denotes TwoScale.

The bypass fraction rose from at most 0.5% for ASPL ≤ 1.75 to 19.75% (95% interval 15.8–24.0%) at ASPL 4 with $k = 1$, and to 32.5% for both widths at ASPL 6. The mean export shift was resolved, with intervals excluding zero, for both widths at every ASPL ≥ 5. Its values ranged from -6.2% to -12.4%; at ASPL 6 they were -8.3% ($[-13.1, -3.5]$) for $k = 1$ and -6.2% ($[-10.3, -2.3]$) for $k = 5$. Below ASPL 5, a shift was resolved only for $k = 5$ at ASPL 3.75, 4.25 and 4.5 (between -3.7% and -5.8%). No positive shift was resolved anywhere in the sweep, in agreement with the direction predicted by Proposition 2.

The irregular centre panel requires care. For nearly exponential export times and $R = 400$ runs per model, the expected e_W of two samples from the same law is about $\sqrt{\pi/R} \approx 9\%$ (Appendix B). The values of 3.7–8.6% observed for ASPL ≤ 1.75 are therefore consistent with pure sampling variability. Single-export distances should thus be read together with the mean shift and its interval. Bypass, by contrast, is zero in Full by construction, so any observed bypass is a genuine discrepancy. At ASPL 4 it affects one fifth of the exports while the export-time distributions remain close.

6.3. Repeated crossings through a chain of cliques

When internal spread is fast, close agreement survives nine successive aggregate crossings. In the chain of ten 1000-node cliques with $k = 1$, the median terminal arrival was 2785 in Full and 2877 in TwoScale, and $e_W = 3.34\%$ (Figure 4). The permutation floor of this cell was 5.1% ($p = 0.32$), so the two terminal distributions are statistically indistinguishable. The CDFs cross, so the difference is not a fixed delay. A 1000-node clique saturates on average within $2H_{999}/(\beta n) \approx 5$ time units, compared with a mean edge-crossing time of 333 ($\rho = 0.002$). The aggregate rate therefore differs from the exact one only during a negligible fraction of each crossing.

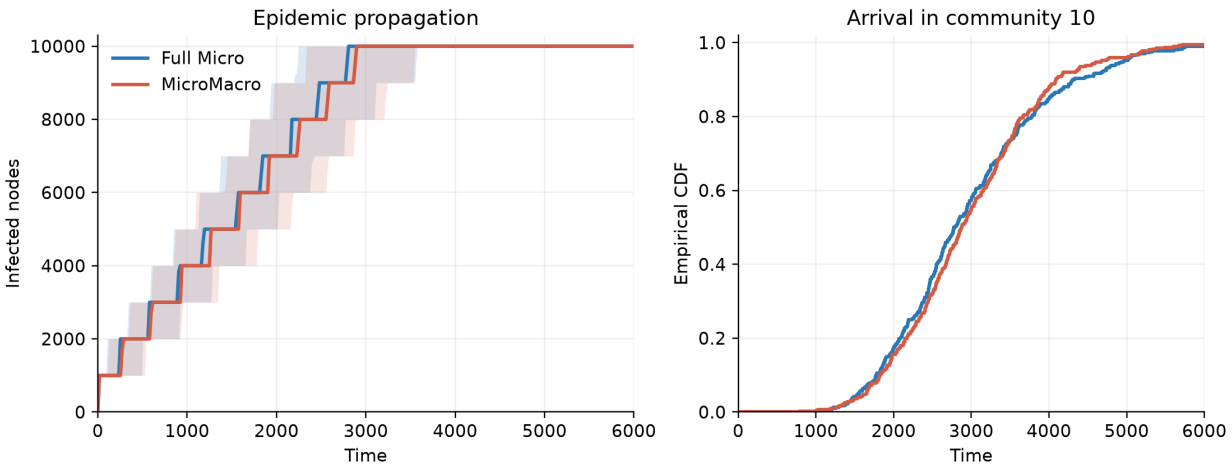


Figure 4: Chain of ten 1000-node cliques with one edge between consecutive cliques ($k = 1$, $\beta = 0.003$, 400 runs per model). Left: pointwise median number of infected nodes with interquartile bands. Right: empirical CDF of the arrival time in the tenth community (T_9). Four FULL and two TWOSCALE arrivals lie beyond the displayed range; all were observed.

6.4. Chains of random communities

Sparse, long-path communities produce systematically early arrivals whose relative size is stable along the chain while the absolute lead accumulates. Figure 5 shows the cumulative arrival times in chains of 1000-node communities. For target ASPL 2–3, terminal e_W was 1.4–2.5%, close to its sampling floor. At ASPL 6, TWOSCALE reached the last community earlier by 13.9% in median for $k = 1$ and by 10.7% for $k = 5$ ($e_W = 13.6\%$ and 10.9%). Along the ASPL-5 row with $k = 1$, the relative median lead was about 10% at both the first and the last community, while its absolute size grew from about 107 to 1018 time units. A constant relative error along a chain therefore means a growing absolute error at distant communities. The positive first-column values in the clique row (+12.6% and +11.5%) are not resolved: for $k = 1$, the first-crossing mean shift is +4.9% with a 95% interval of $[-8.0, 18.3]$%. Single-crossing distances also have a larger sampling floor than cumulative ones.

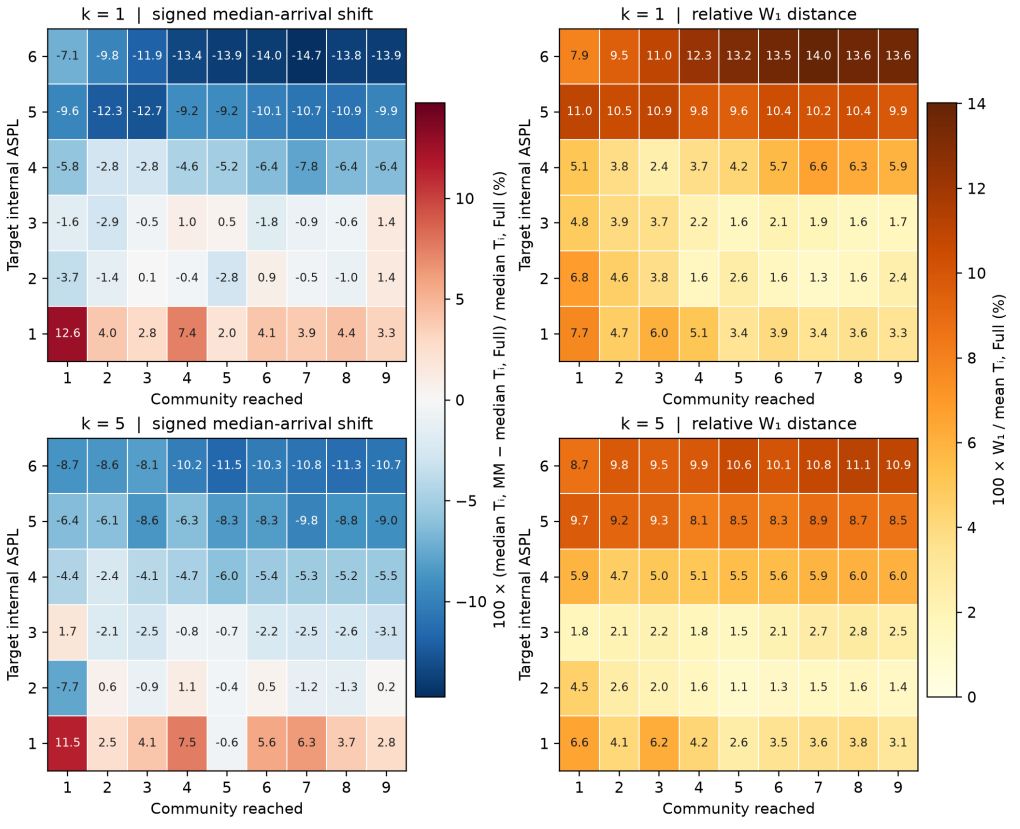


Figure 5: Cumulative arrival accuracy in ten-community chains with $n = 1000$, by target ASPL (rows) and community reached ($c = 1, \ldots, 9$, columns), for $k = 1$ (top) and $k = 5$ (bottom). Left: relative median shift b_c (negative: TwoScale earlier). Right: relative Wasserstein distance. Each cell uses 400 matched runs; no uncertainty is shown (see Figure 6 for the terminal arrivals).

6.5. Community size and interface width

The same internal path length is much less favourable in small communities. Figure 6 reports the terminal discrepancy for all 48 configurations of the size sweep. The permutation floor ranged from 1.6% to 5.9%. In the 16 configurations with ASPL ≤ 2, e_W was 1.1–4.4%, and in 14 of them it was not distinguishable from the floor at the 5% level. In 31 of the 32 configurations with ASPL ≥ 3, e_W exceeded the floor ($p < 0.01$), and the median shift was negative with a block-bootstrap interval excluding zero. The exception was $n = 1000$, ASPL 3, $k = 1$, with $e_W = 1.7\%$. At a fixed ASPL ≥ 3, the discrepancy decreased monotonically with community size for both interface widths. At ASPL 6, e_W fell from 41.3% for $n = 50$ to 13.6% for $n = 1000$ with $k = 1$, and from 32.4% to 10.9% with $k = 5$. Wider interfaces reduced the discrepancy in sparse communities but did not remove it. The fraction of TwoScale crossings that preceded infection of any bridge node of the source community rose from 0.1–2.3% at ASPL 1 to 30–52% at ASPL 6.

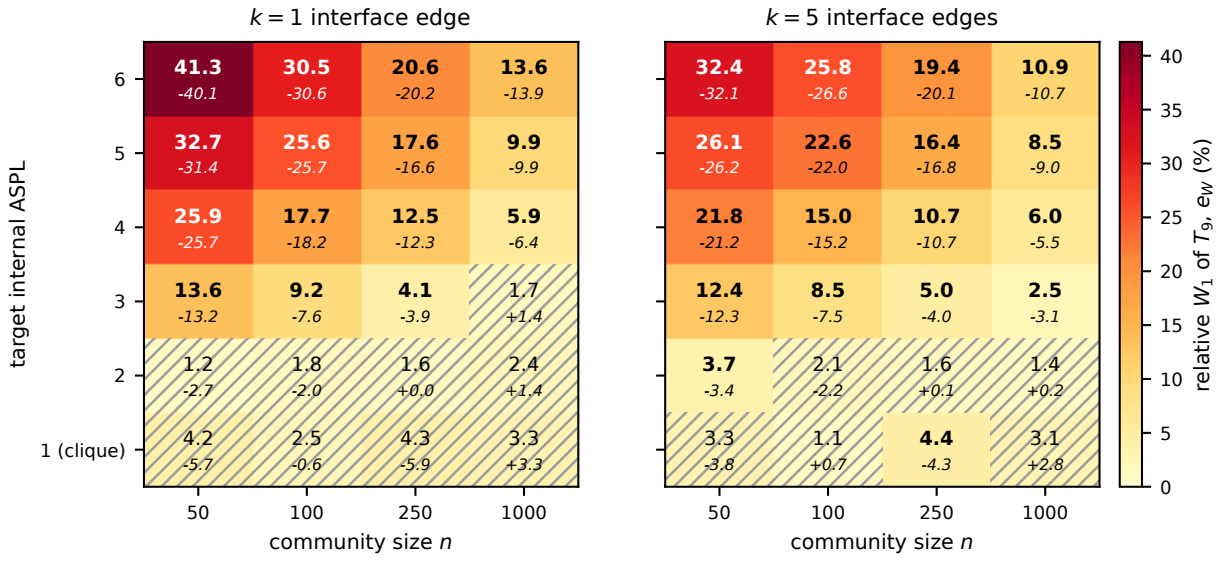


Figure 6: Terminal-arrival accuracy in ten-community chains for all 48 configurations of the size sweep. Colour and bold numbers: relative Wasserstein distance e_W of T_9 (%). Italic numbers: relative median shift b_9 (%; negative means TwoScale earlier). Hatched cells: e_W not distinguishable from its within-pair permutation floor ($p \geq 0.05$). Each cell: 400 matched runs.

6.6. A single time-scale ratio organises the discrepancy

Expressed through $\rho = \beta\sigma_T$, the results of all sizes, structures and interface widths collapse onto one curve. We computed σ_T in each configuration as the standard deviation of node infection times within each source community of the Full runs, averaged over communities and runs. Figure 7(a,b) plots the terminal e_W and b_9 against ρ. The rank correlation between e_W and ρ was 0.92 over all 48 configurations and 0.98 over the 32 configurations with ASPL ≥ 3; the corresponding values for target ASPL were 0.82 and 0.67. In the 18 configurations with $\rho < 0.25$, $e_W \leq 4.4\%$, and 15 were indistinguishable from the permutation floor. All 30 configurations with $\rho \geq 0.25$ were resolved. For $\rho \geq 0.3$, a log–log fit gave $e_W \approx 15\rho\,\%$ (slope 1.03; median relative deviation 8%, maximum 41%), and $b_9 \approx -e_W$, i.e. the discrepancy was almost entirely an early shift. We also examined two alternative scalings: $\beta\sqrt{k}\sigma_T$, suggested by (8), and the mean time from a community's first infection to infection of its first bridge node, multiplied by βk. Both separated the two interface widths more than ρ did.

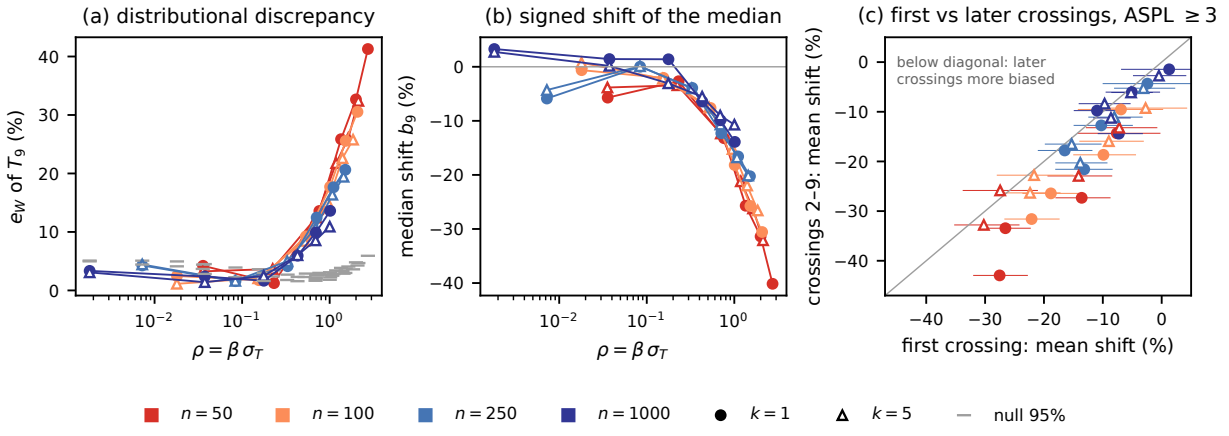


Figure 7: (a) Relative Wasserstein distance and (b) relative median shift of the terminal arrival against $\rho = \beta\sigma_T$, for all 48 chain configurations (colours: community size; filled circles: $k = 1$; open triangles: $k = 5$; lines connect ASPL 1–6 at fixed n and k). Grey ticks in (a): 95th percentile of the within-pair permutation floor of each configuration. (c) Local mean shift of the first crossing against that of crossings 2–9, for configurations with ASPL ≥ 3, with 95% block-bootstrap intervals of the first-crossing shift.

Figure 7(c) separates the crossings. By Remark 1, the first crossing in our design is subject only to source-side and numerical mechanisms, whereas later crossings are also subject to placement after the first import and to surplus imports. For $k = 1$, later crossings were more strongly biased than the first in 15 of the 16 configurations with ASPL ≥ 3. The difference was resolved in 9 of them, including all four sizes at ASPL 6; for example, -27.5% for the first crossing against -42.9% for crossings 2–9 at $n = 50$. For $k = 5$, the difference was smaller (14 of 16 point estimates negative, 3 resolved). This dependence on k is what interface exhaustion predicts: a single physical edge permits one crossing, whereas five edges permit up to five, so surplus aggregate imports matter less. Because seeds are shared between configurations, we regard this pattern as supporting rather than conclusive evidence. A direct count of surplus imports would require event-type logs that were not stored.

7. Discussion

7.1. Practical guidance

The results suggest a simple procedure for deciding whether aggregate coupling is adequate.

Decide which observable matters. For the timing of spread between communities, the two-scale simulator can be accurate. For transmission routes, it cannot be trusted. Up to half of its crossings in sparse communities, and a fifth of exports even where export-time distributions agreed, occurred before any true bridge node was infected. Questions that depend on bridge nodes, such as targeting interventions at the individuals who connect communities or attributing an outbreak to a source, require boundary information that the aggregate interface does not contain.

Estimate the time-scale ratio. Compute $\rho = \beta_{\text{out}}\sigma_T$. Since σ_T describes spread within a single community, it can in principle be estimated from the local engines alone, by simulating an isolated community from a uniformly placed seed. We computed it from the reference runs and have not tested this local estimate. In our benchmarks, terminal arrivals were statistically indistinguishable for $\rho \lesssim 0.25$. For $0.3 \lesssim \rho \lesssim 3$, TwoScale arrived early by roughly $15\rho\%$. This rule is an empirical summary of one family of synthetic networks and parameters, not a general error law.

Check absolute as well as relative errors. Along a chain, the relative discrepancy was nearly constant, so the absolute lead at distant communities grew in proportion to the travel time. For intervention timing at a specific destination, the absolute discrepancy at that destination is the relevant quantity.

Calibrate distances against sampling variability. With a few hundred runs per model, distances of a few percent between arrival-time distributions are expected even between identical models (Appendix B). Permutation baselines or analytical floors should accompany such comparisons.

7.2. Computational cost

The benchmarks were designed for accuracy assessment, so their explicit interfaces contain at most five edges and the full reference is cheap. In this regime, aggregation removes almost no work, and the cost of grid synchronisation dominates. The median wall-clock time per replicate of the Python implementations ranged from 0.04–0.07 s (Full) and 0.6–31 s (TwoScale) for $n = 50$ to 8.6–18 s and 6.7–124 s for $n = 1000$. For $n = 1000$, the ratio of the two medians ranged from 0.76 (ASPL 2, $k = 5$) to 6.9 (ASPL 6, $k = 1$). Two conclusions follow. First, in settings where the full network is known and small, Full should be preferred. Second, the overhead of the two-scale method stems from the black-box scheduler rather than from aggregation. An event-driven scheduler that integrates the piecewise-constant hazard exactly (Section 3.5) would remove both this overhead and the synchronisation error. Computational gains are expected when inter-community contacts are numerous or unavailable, which is the setting the method is intended for (Kuryliak and Emmerich, 2025); quantifying them is left to future work.

7.3. Limitations

The experiments use SI dynamics. With recovery, bridge nodes can stop transmitting while the community-wide fraction remains positive, and the aggregate rate may then also overestimate exports late in a local outbreak. All networks are synthetic, the macro-topology is a chain, and internal and external per-edge rates are equal. Interfaces were placed at random, which by Remark 1 neutralises placement error for first imports. Structured interfaces, for example with bridges at hubs or with incoming and outgoing bridges far apart, may behave differently. The synchronisation step was not varied, so the numerical component of the discrepancy was not isolated; Proposition 1 provides a direct way to do so. Finally, ρ was evaluated with reference runs and chosen among a small number of candidate scalings; its predictive value for other network families remains to be tested.

7.4. Possible extensions

The analysis points to modifications that use only aggregate information. Interface exhaustion can be represented by limiting the number of imports per bundle to its number of channels, which is known whenever

W_{ij} counts edges, or by reducing λ_{ij} as this capacity is consumed. When partial boundary information is available, $\bar{y}_i$ in (4) can be replaced by the infected fraction among known bridge nodes. Placement can then be restricted to known receiving endpoints. Each such variant can be evaluated against FULL and against exact simulation on $\tilde{G}$, which separates modelling from numerical error.

8. Conclusions

The two-scale method couples exact local Gillespie engines through aggregate inter-community hazards. In its ideal form, it is exact simulation on a surrogate network in which every interface is smeared uniformly over all pairs of nodes. This identifies what the method loses: the state of the actual bridge nodes, the location of imports and the finite capacity of thin interfaces. For randomly placed interfaces, it also implies that aggregate export clocks are stochastically early. In controlled comparisons with full-network Gillespie simulation, terminal arrival-time distributions were statistically indistinguishable when infection spread through communities quickly relative to the time needed to cross an interface edge. As the internal timing spread approached and exceeded that crossing time, the two-scale simulator arrived systematically early, by up to 40% in small sparse communities. The ratio $\rho = \beta\sigma_T$ organised these results across community sizes, internal structures and interface widths. Close timing never guaranteed admissible transmission routes. The method is therefore a reasonable tool for propagation-time questions when ρ is small. It is not suited to questions about individual transmission routes, or to sparse communities with long internal paths unless the interface is refined.

A. Clique formulas and verification

Consider an n-node clique with SI dynamics, per-edge rate β, a uniformly placed seed and a uniformly placed set of k bridge nodes. When m nodes are infected, the next internal infection occurs at rate $\beta m(n-m)$, independently of which nodes are infected, and the infected set is a uniform m-subset. The probability that none of the first m infected nodes is a bridge is $\binom{n-k}{m}/\binom{n}{m}$, hence

$$\mathbb{E}[T_B] = \sum_{m=1}^{n-k} \frac{\binom{n-k}{m}}{\binom{n}{m}} \frac{1}{\beta m(n-m)}, \qquad \mathbb{E}[T_B] = \frac{H_{n-1}}{\beta n} \quad (k=1), \tag{11}$$

where H_{n-1} is the harmonic number; the case of a bridge seed ($T_B = 0$) is included. For $k = 1$, the export time in Full is T_B plus an independent $\mathrm{Exp}(\beta)$ waiting time. The expected time to infect the whole clique is $2H_{n-1}/(\beta n)$. Table 3 compares (11) with the first community of the clique chains.

n	k	$\mathbb{E}[T_B]$, Eq. (11)	FULL mean T_B	FULL wait after T_B
50	1	29.86	29.72 ± 0.71	354.0 ± 16.9
100	1	17.26	17.05 ± 0.38	326.5 ± 17.0
250	1	8.13	7.84 ± 0.15	347.4 ± 16.6
1000	1	2.49	2.51 ± 0.04	353.8 ± 17.9
50	5	15.97	15.96 ± 0.55	–
100	5	10.31	9.64 ± 0.29	–
250	5	5.35	5.16 ± 0.12	–
1000	5	1.80	1.77 ± 0.03	–

Table 3: Verification of the reference simulator on the first community of the clique chains ($\beta = 0.003$, 400 runs per cell): mean bridge-activation time and, for $k = 1$, mean wait after activation (exact value $1/\beta = 333.3$). Values are mean $\pm$ standard error.

B. Sampling floor of the Wasserstein distance

Let F_R and G_R be empirical CDFs of two independent samples of size R from the same continuous law F. Then $W_1 = \int |F_R - G_R|\, dx$, and $\sqrt{R}\,(F_R - G_R)$ is approximately $\sqrt{2}$ times a Brownian bridge evaluated at $F(x)$ (del Barrio et al., 1999). Since $\mathbb{E}|B(u)| = \sqrt{2u(1-u)/\pi}$ for a Brownian bridge B,

$$\mathbb{E}[W_1] \approx \frac{2}{\sqrt{\pi R}} \int \sqrt{F(x)\big(1 - F(x)\big)}\, dx.$$

For an exponential law with mean μ, the integral equals $\mu\pi/2$, so that $\mathbb{E}[W_1]/\mu \approx \sqrt{\pi/R}$, which is 8.9% for $R = 400$ and 1.8% for $R = 10\,000$. Arrival times that are sums of several crossings are less dispersed relative to their mean, and their floor is correspondingly lower; the permutation floors of the chain configurations ranged from 1.6% to 5.9%.

C. Reproducibility details

Seeds are derived deterministically from the experiment parameters. Each TwoScale engine draws from a private pseudorandom stream; midpoint probes copy the stream state of their engine and do not advance it. Two Full runs ($n = 50$, ASPL 6, $k = 1$) had not reached the tenth community by $T = 30\,000$. They were rerun to $T = 40\,000$ with the same graphs, initial nodes and event seeds. Their event histories up to 30 000 were verified to match the saved runs, and their terminal arrivals occurred at about 31 059 and 31 007. No horizon value was substituted for an unobserved event. The permutation and bootstrap analyses of the chain configurations, the descriptor σ_T and Figures 6–7 were computed from the saved runs by the scripts `extrascripts/analysis/smpt_twoscale_paper_metrics.py` and `extrascripts/visualization/smpt_twoscale_paper_figures.py`.

CRediT authorship contribution statement

Yulian Kuryliak: Conceptualization, Methodology, Software, Investigation, Formal analysis, Visualization, Writing – original draft. **Michael T. M. Emmerich:** Conceptualization, Methodology, Supervision, Writing – review & editing. [Authors to confirm.]

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

[Funding information to be added by the authors.]

Data availability

The simulation code, saved simulation outputs and analysis scripts are available from the corresponding author on reasonable request. [Replace with a repository DOI if the data are deposited.]

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used [name of tool or service] in order to [reason]. After using this tool or service, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. [Authors to complete.]

References

Ajelli, M., Gonçalves, B., Balcan, D., Colizza, V., Hu, H., Ramasco, J.J., Merler, S., Vespignani, A., 2010. Comparing large-scale computational approaches to epidemic modeling: Agent-based versus structured metapopulation models. BMC Infectious Diseases 10, 190. doi:10.1186/1471-2334-10-190.

Anderson, D.F., 2007. A modified next reaction method for simulating chemical systems with time dependent propensities and delays. The Journal of Chemical Physics 127, 214107. doi:10.1063/1.2799998.

Balcan, D., Colizza, V., Gonçalves, B., Hu, H., Ramasco, J.J., Vespignani, A., 2009. Multiscale mobility networks and the spatial spreading of infectious diseases. Proceedings of the National Academy of Sciences 106, 21484–21489. doi:10.1073/pnas.0906910106.

Balcan, D., Colizza, V., Gonçalves, B., Hu, H., Ramasco, J.J., Vespignani, A., 2010. Modeling the spatial spread of infectious diseases: The global epidemic and mobility computational model. Journal of Computational Science 1, 132–145. doi:10.1016/j.jocs.2010.07.002.

Ball, F., Mollison, D., Scalia-Tomba, G., 1997. Epidemics with two levels of mixing. The Annals of Applied Probability 7, 46–89.

Barbosa, H., Barthelemy, M., Ghoshal, G., James, C.R., Lenormand, M., Louail, T., Menezes, R., Ramasco, J.J., Simini, F., Tomasini, M., 2018. Human mobility: Models and applications. Physics Reports 734, 1–74. doi:10.1016/j.physrep.2018.01.001.

del Barrio, E., Giné, E., Matrán, C., 1999. Central limit theorems for the Wasserstein distance between the empirical and the true distributions. The Annals of Probability 27, 1009–1071.

Cao, Y., Gillespie, D.T., Petzold, L.R., 2005. The slow-scale stochastic simulation algorithm. The Journal of Chemical Physics 122, 014116.

Chang, S., Pierson, E., Koh, P.W., Gerardin, J., Redbird, B., Grusky, D., Leskovec, J., 2021. Mobility network models of COVID-19 explain inequities and inform reopening. Nature 589, 82–87. doi:10.1038/s41586-020-2923-3.

Colizza, V., Pastor-Satorras, R., Vespignani, A., 2007. Reaction–diffusion processes and metapopulation models in heterogeneous networks. Nature Physics 3, 276–282. doi:10.1038/nphys560.

Colizza, V., Vespignani, A., 2008. Epidemic modeling in metapopulation systems with heterogeneous coupling pattern: Theory and simulations. Journal of Theoretical Biology 251, 450–467. doi:10.1016/j.jtbi.2007.11.028.

Danon, L., Ford, A.P., House, T., Jewell, C.P., Keeling, M.J., Roberts, G.O., Ross, J.V., Vernon, M.C., 2011. Networks and the epidemiology of infectious disease. Interdisciplinary Perspectives on Infectious Diseases 2011, 284909. doi:10.1155/2011/284909.

Ethier, S.N., Kurtz, T.G., 1986. Markov Processes: Characterization and Convergence. Wiley, New York.

Eubank, S., Guclu, H., Kumar, V.S.A., Marathe, M.V., Srinivasan, A., Toroczkai, Z., Wang, N., 2004. Modelling disease outbreaks in realistic urban social networks. Nature 429, 180–184. doi:10.1038/nature02541.

Gibson, M.A., Bruck, J., 2000. Efficient exact stochastic simulation of chemical systems with many species and many channels. The Journal of Physical Chemistry A 104, 1876–1889.

Gillespie, D.T., 1977. Exact stochastic simulation of coupled chemical reactions. The Journal of Physical Chemistry 81, 2340–2361. doi:10.1021/j100540a008.

Gomes, C., Thule, C., Broman, D., Larsen, P.G., Vangheluwe, H., 2018. Co-simulation: A survey. ACM Computing Surveys 51, 49:1–49:33.

Haseltine, E.L., Rawlings, J.B., 2002. Approximate simulation of coupled fast and slow reactions for stochastic chemical kinetics. The Journal of Chemical Physics 117, 6959–6969.

Keeling, M.J., Eames, K.T.D., 2005. Networks and epidemic models. Journal of The Royal Society Interface 2, 295–307. doi:10.1098/rsif.2005.0051.

Kerr, C.C., Stuart, R.M., Mistry, D., Abeysuriya, R.G., Rosenfeld, K., Hart, G.R., Núñez, R.C., Cohen, J.A., Selvaraj, P., Hagedorn, B., George, L., Jastrzebski, M., Izzo, A.S., Fowler, G., Palmer, A., Delport, D., Scott, N., Kelly, S.L., Bennette, C.S., Wagner, B.G., Chang, S.T., Oron, A.P., Wenger, E.A., Panovska-Griffiths, J., Famulare, M., Klein, D.J., 2021. Covasim: An agent-based model of COVID-19 dynamics and interventions. PLoS Computational Biology 17, e1009149. doi:10.1371/journal.pcbi.1009149.

Kiss, I.Z., Miller, J.C., Simon, P.L., 2017. Mathematics of Epidemics on Networks: From Exact to Approximate Models. Springer, Cham. doi:10.1007/978-3-319-50806-1.

Kuryliak, Y., Emmerich, M., 2025. Towards complexity reduction of large-scale epidemic simulation in two-scale networks, in: Proceedings of the Modern Machine Learning Technologies Workshop (MoMLeT 2025), pp. 327–336. URL: https://ceur-ws.org/Vol-4004/.

Kuryliak, Y., Emmerich, M.T.M., Dosyn, D., 2025. Simulating epidemic peak dynamics on complex networks using efficient gillespie algorithms. Infection, Genetics and Evolution 132, 105768. doi:10.1016/j.meegid.2025.105768.

Pastor-Satorras, R., Castellano, C., Van Mieghem, P., Vespignani, A., 2015. Epidemic processes in complex networks. Reviews of Modern Physics 87, 925–979. doi:10.1103/RevModPhys.87.925.

Shaked, M., Shanthikumar, J.G., 2007. Stochastic Orders. Springer, New York.

Vestergaard, C.L., Génois, M., 2015. Temporal gillespie algorithm: Fast simulation of contagion processes on time-varying networks. PLoS Computational Biology 11, e1004579. doi:10.1371/journal.pcbi.1004579.